Design of a hardware accelerator for autonomous driving

Sparsh Mittal

ADV Design Summary

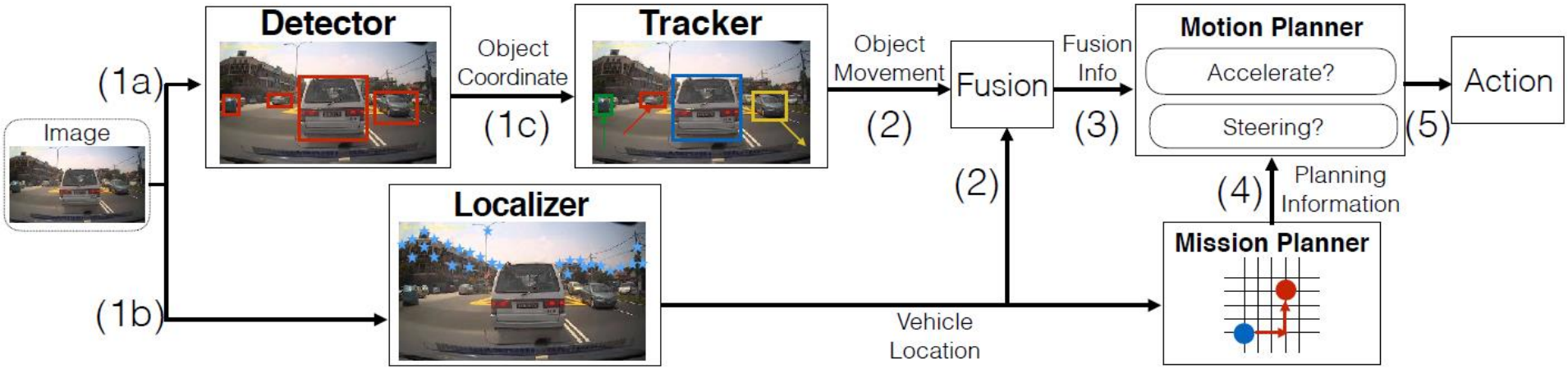
- **Goal**: drive the vehicle to reach the destination
- **Sensors:** video cameras, laser scanners, and milliwave radars, etc.
- 3 major **tasks**:

(1) scene recognition for localizing the vehicle at 0.1m-level and tracking nearby objects

(2) path planning for generating the future paths

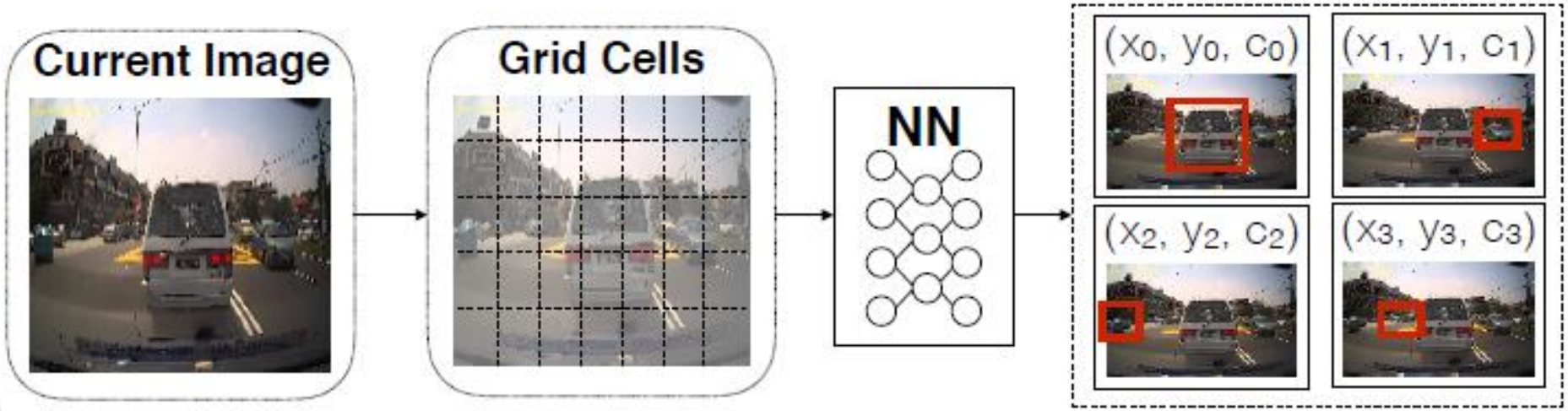
(3) vehicle control for operating the vehicle to follow the planned paths

Overview of working of ADVs



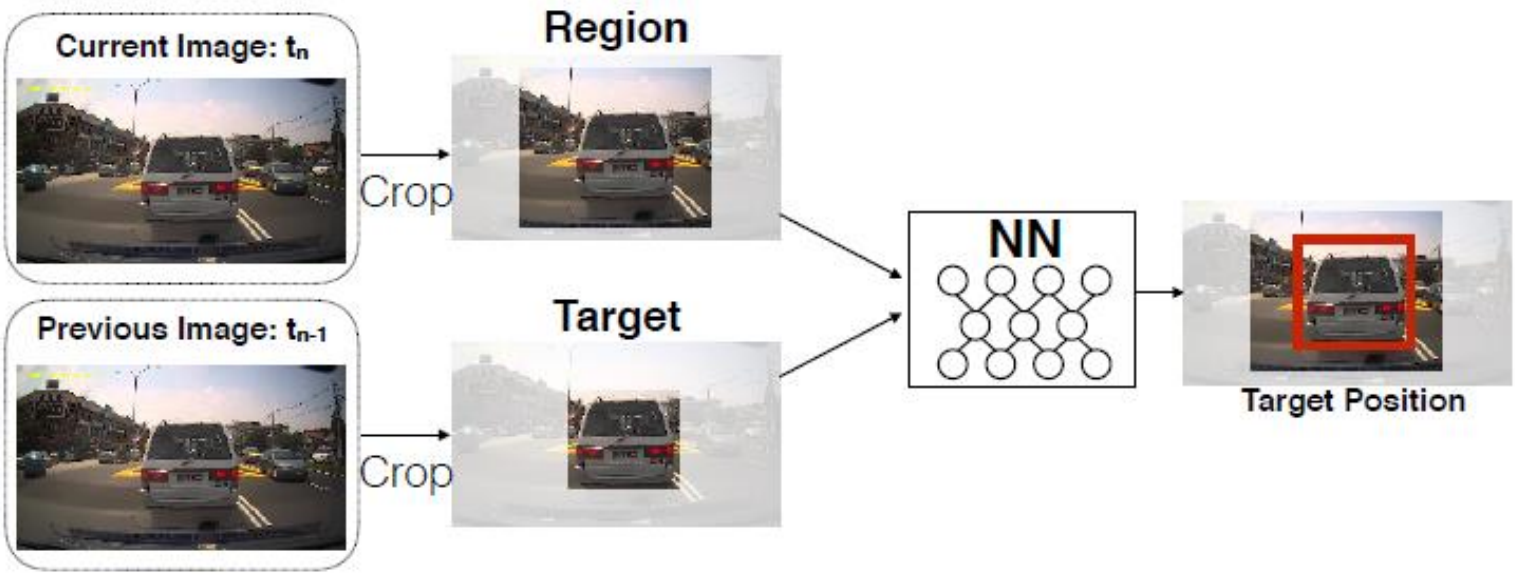
Object Detection

- They use YOLO algorithm (accurate and fast)
- Partition input image into sub-regions
- Predict coordinates of detected objects and the confidence for each sub-region using DNN
- Focus on 4 categories: vehicles, bicycles, traffic signs and pedestrians



Object tracking

- They use GOTURN algorithm
- Crop current image into a search region
- Crop previous image into the tracking target only based on the previous result
- Feed both search region and target to NN which generates the bounding box of target to be tracked in the current image



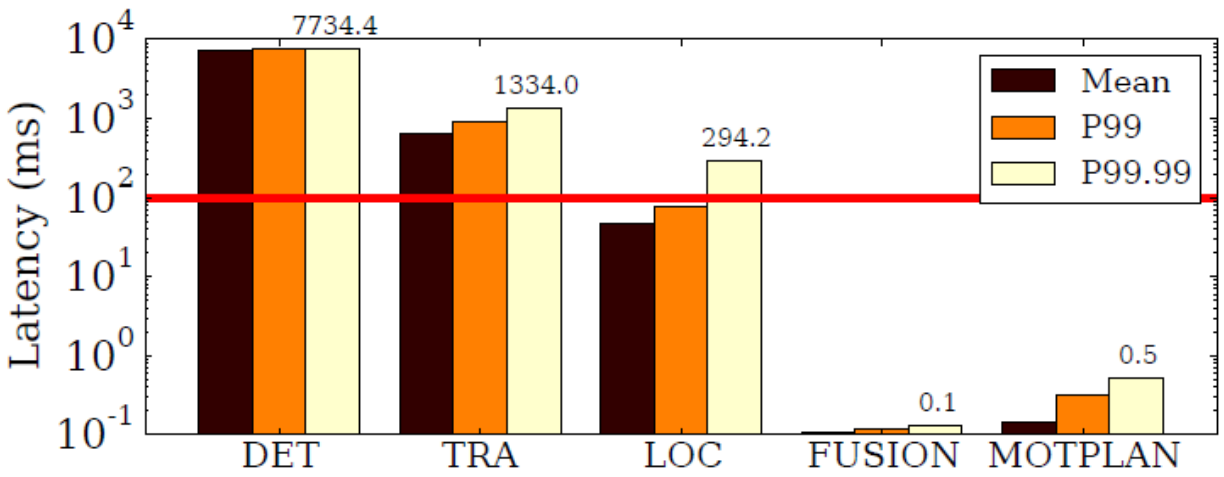
Localization

- They use ORB-SLAM algorithm
- Detect feature points from the video stream
- Generate descriptions of the extracted feature points
- Match the current descriptions with the prior map database to localize the vehicle position
- Update the map if surroundings are different from that in prior-map

Other steps: motion planning (not discussed here)

Results

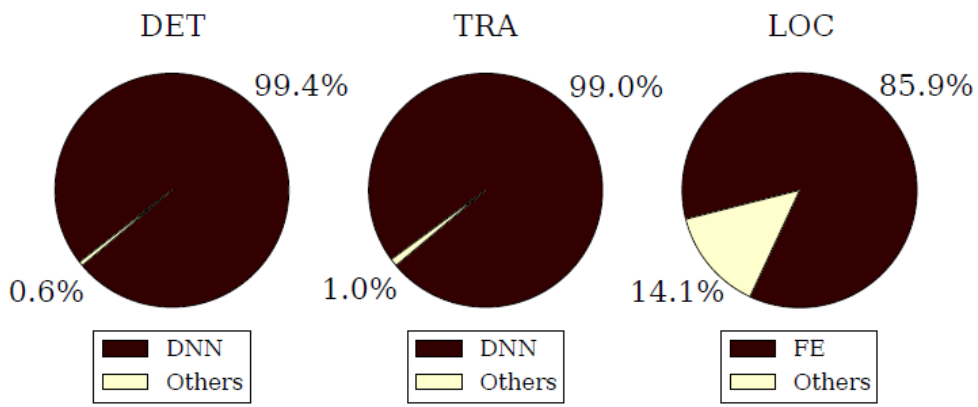
Latency of these components on Multicore CPU



DET = detection
TRA = tracking
LOC = localization

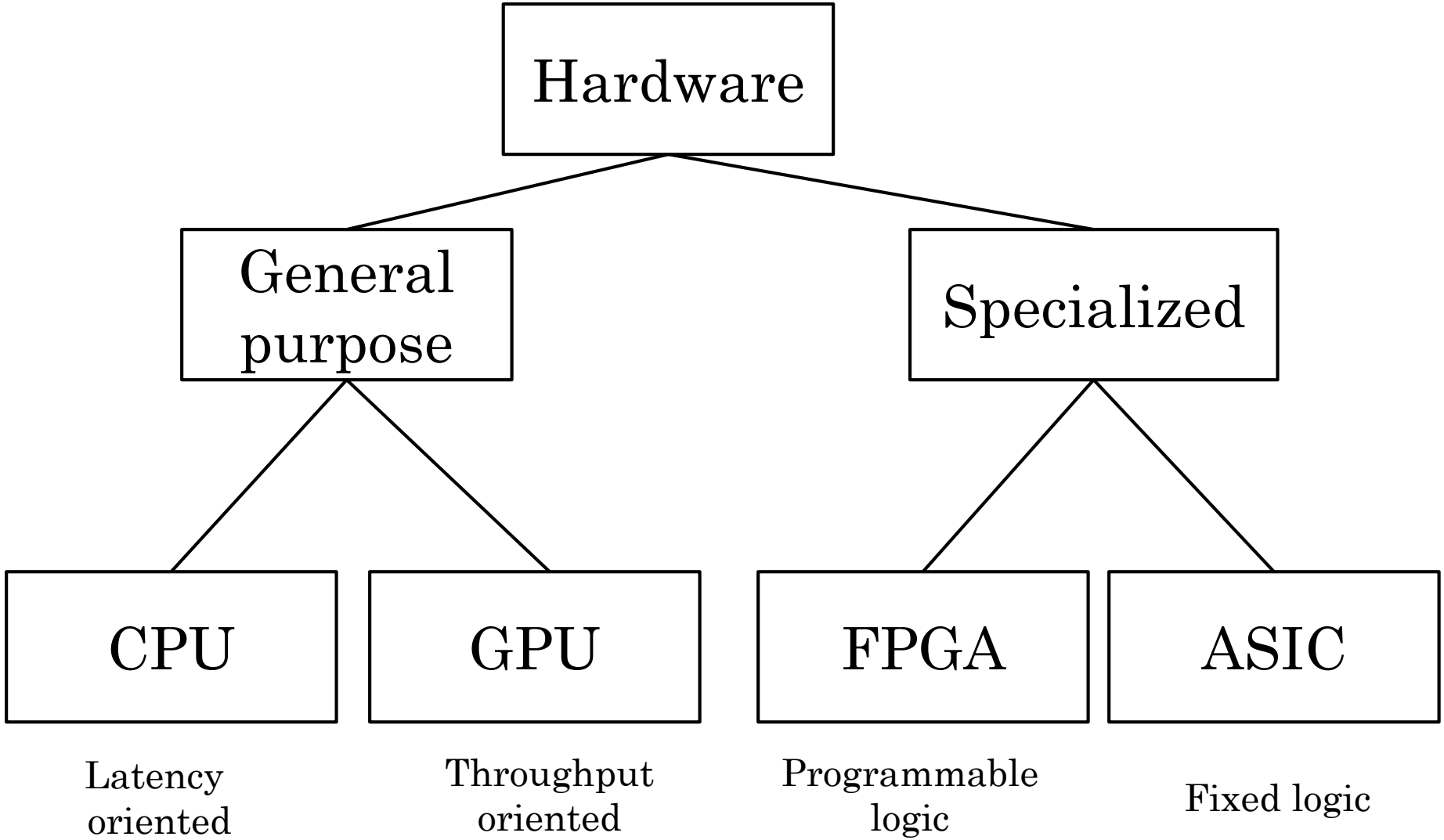
- DET, TRA and LOC dominate overall latency
- Latency of DET, TRA and LOC already exceeds 100ms latency constraint
- CPU: not ideal for meeting real-time deadlines

Cycle-breakdown of DET, TRA and LOC



FE = feature extraction

- Candidate for acceleration:
  - DNN in DET and TRA
  - FE in LOC
- We accelerate them using GPU, FPGA and ASIC


Hardware
General purpose
Specialized
CPU
GPU
FPGA
ASIC
Latency oriented
Throughput oriented
Programmable logic
Fixed logic

Computing platforms

* = DSPs

	CPU	GPU	FPGA	ASIC (CNN)	ASIC (FC)	ASIC (LOC)
Model	Intel Xeon E5-2630 v3	NVIDIA TitanX (Pascal)	Altera Stratix V	TSMC 65 nm	TSMC 45 nm	ARM 45 nm
Frequency	3.20 GHz	1.4 GHz	800 MHz	200 MHz	800 MHz	4 GHz
# Cores	16	3584	256*	N/A	N/A	N/A
Memory	128 GB	12 GB	2 GB	181.5 KB	N/A	N/A
Memory BW	59.0 GB/s	480.0 GB/s	6.4 GB/s	N/A	N/A	N/A

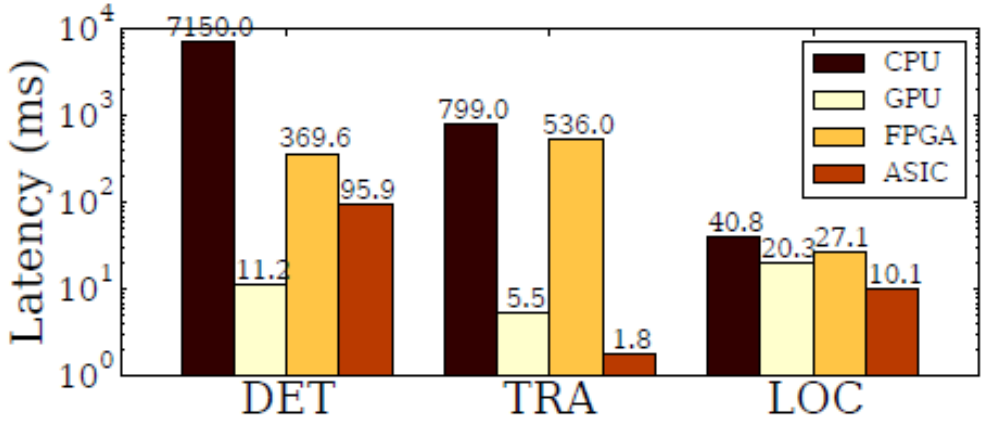
A single camera is assumed

Findings:

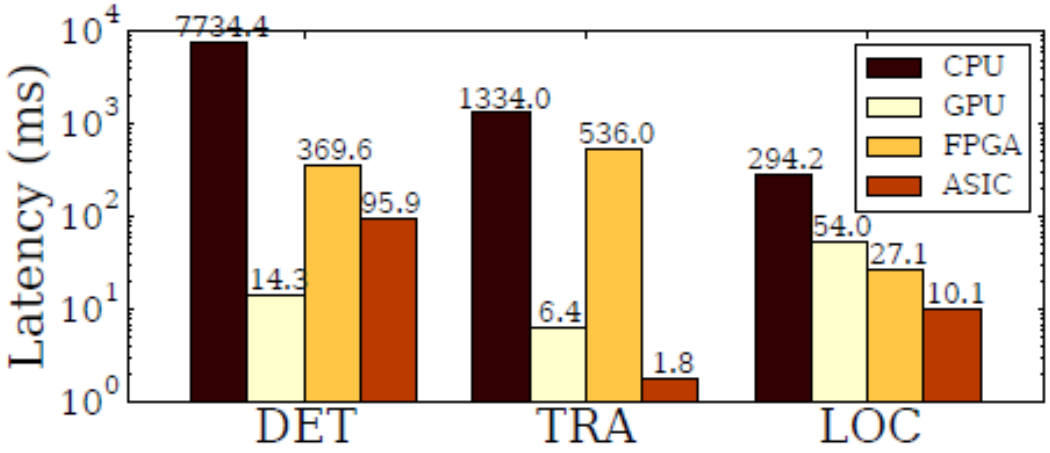
- Mean latency can lead to misleading conclusions => use tail latency to evaluate predictability
- GPU, FPGA and ASIC reduce tail latency by 169×, 10×, and 93× respectively compared to CPU.

Results on Latency

Mean Latency Across Platforms



99.99th-Percentile Latency Across Platforms



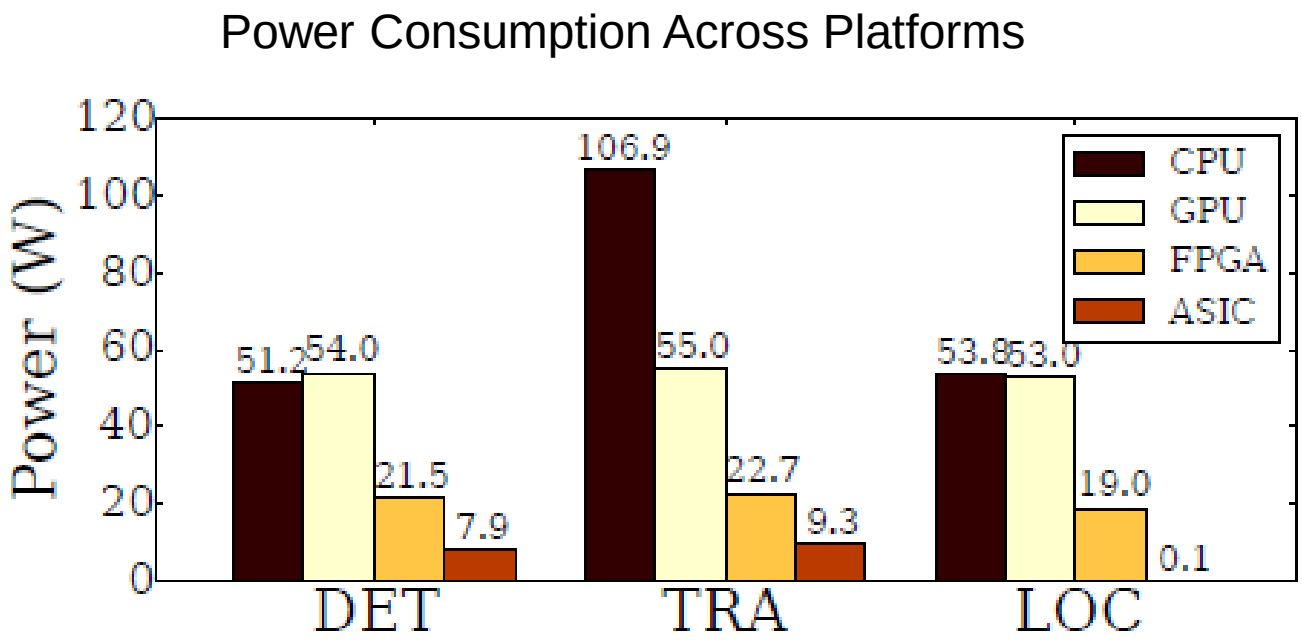
Latency of running DET or TRA alone on CPUs or FPGAs has already exceeded the end-to-end latency constraints,

Results

- Multicore CPUs can execute localization algorithm within the performance constraints on average (i.e., mean latency), they have high tail latency across all three tasks.
- Due to large performance variability, tail latency should be used to evaluate the performance

- The other computing platforms do not experience significant increase from mean latency to tail latency =>
- they are preferable for such mission-critical real-time applications

Results on Power



- FPGA and ASIC power << CPU/GPU power
- Choice of accelerator platform has a significant impact on the vehicle driving range and fuel efficiency.

Finding Summary

- CPU is slowest and also has highest unpredictability
- FPGAs cannot meet performance targets due to limited number of DSPs
- FPGA and ASIC much more energy efficient than GPU
- GPUs: fast and meets timing deadline, but is power hungry. It reduces driving range by 12%
- FPGAs and ASIC reduce driving range by <5%.
- ASIC provides high predictability and can meet real-time processing targets of ADV.
- Limited computation capability remains bottleneck in benefitting from multiple/high-resolution cameras